

4-11

security

Definita come un insieme di azioni per rendere sicure le reti e i servizi. Deve essere garantita in tutti gli aspetti del sistema, dal fisico all'user management.

X800 è uno standard che definisce un approccio sistematico alla sicurezza di un sistema.

Un sistema sicuro ha 6 temi in cui investire:

- data availability
- data authentication
- data integrity
- access control
- non-repudiation
- anonymity

I meccanismi di sicurezza sono funzioni che possono aiutare a ottenere i servizi di sicurezza. Gran parte dei meccanismi si basa sulla cryptography. Essi sono principalmente:

- encryption
- digital signature
- access control
- integrity mechanisms
- authentication protocols



I sistemi non possono essere sicuri al 100%. Un sistema sicuro è un sistema che resiste a un determinato attaccante.

Per questo è necessario studiare la storia degli attacchi passati. Senza questo sforzo non possiamo progettare meccanismi di sicurezza adeguati per realizzare i servizi richiesti.

service availability



Un servizio deve essere utilizzabile sempre. Proteggersi completamente da un attacco *DoS* non è possibile.

La disponibilità di un servizio è violata se si è vittima di un *DoS* (Denial of Service Attack). La disponibilità è la più difficile da garantire, perché esistono sempre risorse fisiche che l'attaccante può saturare.

Per scongiurare un attacco *DoS* è necessario rendere la saturazione del sistema troppo costosa e quindi non conveniente.

data confidentiality



I dati scambiati devono rimanere confidenziali tra le due entità che partecipano allo scambio.

Una rete Ethernet permette agli altri utenti di *sniffare* i pacchetti di altri PC. Per ottenere confidenzialità è necessario un algoritmo di encryption. Inoltre i dati devono arrivare a destinazione senza essere modificati. L'integrità si ottiene con meccanismi di **hashing**.

authentication

Divisa in 2:

Data Origin Authentication: chi riceve i dati deve essere sicuro che l'entità che li invia sia effettivamente chi dichiara di essere.

Peer Entity Authentication: chi riceve le informazioni deve essere sicuro che la fonte sia quella attesa, non un altro computer nella stessa rete.

L'autenticazione dei dati è ottenuta attraverso la **digital signature**.

access control



L'accesso al servizio deve essere ristretto alle persone autorizzate a fruirne.

Tutte le entità che provano ad accedere al servizio devono identificarsi o autenticarsi. I diritti di accesso possono essere definiti granularmente.

Perdere l'access control comporta che qualcun altro possa usare le tue risorse per attaccare un terzo. L'host da cui l'attacco viene eseguito è sempre coinvolto nelle azioni illegali subite dalle vittime.

non-repudiation



Previene che il mittente o il destinatario possano negare un messaggio trasmesso. Quando un messaggio è inviato, il destinatario ottiene una prova che il messaggio è stato effettivamente inviato dal mittente dichiarato. Allo stesso modo, quando un messaggio è ricevuto, il mittente può provare che il destinatario è quello corretto.

Le digital signatures sono usate come meccanismo per questo servizio.

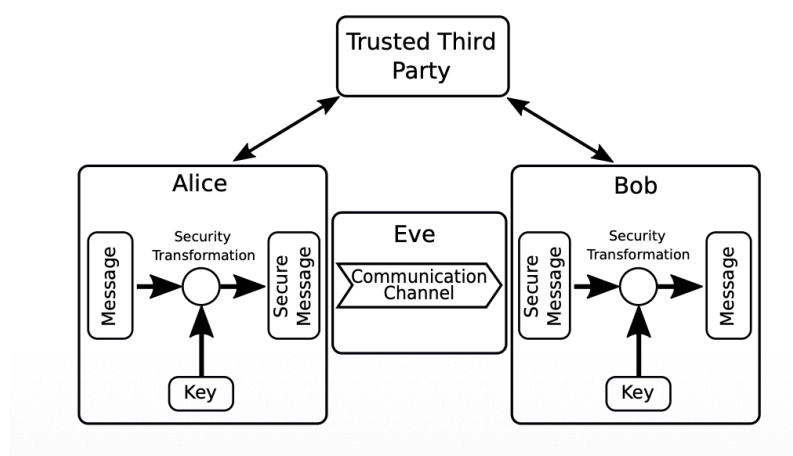
anonymity



La capacità di usare un sistema senza che esso sia in grado di identificare il mittente.

Esistono servizi di anonymity, ad esempio *Tor*.

system model



Questa astrazione può essere usata per ogni sistema, in ogni situazione, ad esempio un sito o un'app, con:

- un mittente e un destinatario
- qualcuno che tenta di interferire con la comunicazione
- un canale
- security transformations
- un terzo ente coinvolto

Assumiamo che i meccanismi di sicurezza siano robusti e che l'attaccante non possa violarli; la cryptography funziona se si usano le funzioni giuste.

Il terzo ente serve a generare, distribuire o certificare le credenziali di mittente e destinatario. Può essere coinvolto direttamente o indirettamente nella comunicazione.

Il flusso di azioni per ottenere un sistema sicuro è il seguente:

Identificare gli attori (mittente e destinatario).

Identificare gli asset che vogliamo proteggere e il loro valore.

Decidere quali security services offrire.

Definire un attacker model e decidere quali rischi accettare.

Scegliere i meccanismi tecnici necessari per ottenere i security services.

Iteriamo continuamente questo processo.



La sicurezza è sempre un trade-off: è necessario valutare il valore del proprio sistema, i security services necessari, i modi in cui può essere attaccato e fare un'analisi dei rischi.

cryptography

La cryptography deve fornire **data integrity**, **secrecy/confidentiality** e **non-repudiation**. Con diverse funzioni crittografiche si ottengono **peer authentication** e **origin authentication**, **access control** e **anonymity**:

- cryptographic algorithm: sequenza di operazioni matematiche applicate a un messaggio M
- cryptographic function: un blocco di codice che implementa l'algoritmo per offrire un meccanismo di sicurezza
- secure protocol: una convenzione tra due entità che comunicano tra loro, che definisce in che modo sono scambiati i dati per garantire determinati servizi

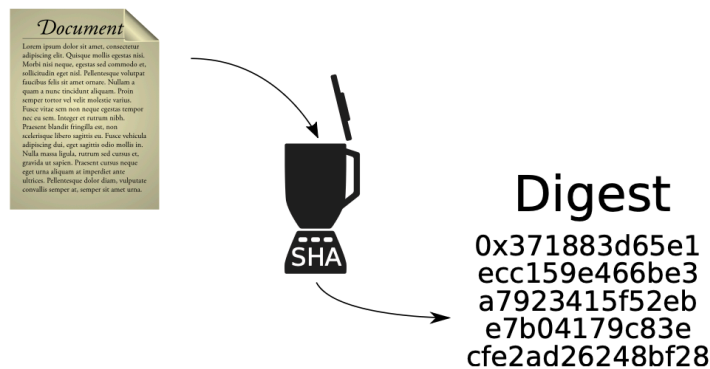
golden rules

1. Non usare mai algoritmi di encryption non noti.
2. Non usare funzioni di encryption closed-source.
3. Utilizza solo le funzioni più popolari e più usate.
4. Usa protocolli standard, non reinventare la ruota.
5. Restringi il servizio, se possibile, agli utenti che supportano la versione più aggiornata del protocollo.

hash functions

Una hash function è una funzione unidirezionale che, applicata a dei dati, genera un **digest**: una stringa di lunghezza prefissata. Diversi input danno origine a diversi **digest**.

Plain Text



Esempio:

Quando Ubuntu rilascia una nuova versione del sistema operativo e poi la distribuisce anche su reti P2P, chi scarica l'immagine da un sito diverso da quello ufficiale può calcolare localmente il **digest** e confrontarlo con quello fornito dal sito ufficiale di Ubuntu (256 bit).



Poiché il dominio delle hash functions (spazio degli input, possibilmente illimitato) è molto più grande del codominio (spazio di output prefissato), ammettiamo **collisions**: $H(M) = H(M') = D$.

- Dato D , trovare X tale che $H(X) = D$ deve essere computazionalmente impossibile.
- Weak collision resistance: dato X , trovare Y tale che $H(X) = H(Y)$ deve essere computazionalmente impossibile.
- Strong collision resistance: trovare due X e Y tali che $H(X) = H(Y)$ deve essere computazionalmente impossibile.

“Computazionalmente impossibile” indica un problema per cui non esiste un algoritmo di complessità polinomiale noto per risolverlo.

Una hash function non è invertibile: poiché D ha dimensione limitata, molte informazioni sono perse. Considerando anche le collisions, trovando un X tale che $H(X) = D$ non sapremo mai se si tratti di M o di un M' .

brute force

Volendo trovare M conoscendo D , posso provare diversi M' finché non trovo una collisione.

Se l'hash ha dimensione 256, ogni tentativo ha probabilità di successo $\frac{1}{2^{256}}$: computazionalmente impossibile.

guided brute force

Definiamo $d(D, D') \in [0, 1]$ una funzione di distanza tra due stringhe binarie, con $d(D, D') = 0 \Leftrightarrow D = D'$.

Supponiamo che Eve parta da M' e applichi piccole modifiche, ottenendo M'' con $D'' = H(M'')$, in modo che $d(D, D'') < d(D, D')$, fino a trovare M .

Perché questo sia possibile dovrebbe valere che, se $d(D, D'') < d(D, D')$, allora anche $d(M, M'') < d(M, M')$: in altre parole, la distanza dovrebbe essere "invariante" rispetto a $H()$.

Questo renderebbe la probabilità di indovinare a ogni step superiore a $\frac{1}{2^{256}}$.

La weak collision resistance previene ciò:

- se l'input è modificato anche di poco, la differenza nel digest deve essere grande
- l'output deve essere imprevedibile; in pratica $H()$ deve comportarsi come un pseudorandom generator

La strong collision resistance richiede di trovare due messaggi che collidono. Per il birthday paradox è più facile: richiede circa 2^{128} tentativi.

- Essendo più facile, è una proprietà più forte.
- Ogni digest ha una probabilità teorica di collision > 0 .
- Ma la consideriamo computazionalmente impossibile da ottenere in pratica.